

ECOM EXPERTS

Quiniela engine

Manual de usuario, técnico y de desarrollo

quiniela-engine · 08/09/2026

Contenido

1. Ponerla en marcha (una sola vez)

2. La pantalla principal: Semana

La última columna: 1ª y 2ª parte reales

Los partidos de ayer aparecen sin resultado

La etiqueta «post-inicio»

3. Los botones de arriba

4. Ficha de un partido

Lo primero: el desglose por tiempos

Las ayudas (?)

Los colores

5. Rendimiento

6. Backtest

7. Staking (Kelly) — léelo entero

8. Modelo

9. Preguntas frecuentes

Instalación en Windows (3 pasos)

Qué ves en la aplicación

Cómo funciona

Datos

Modelo

Tiempos

Evaluación

Backtest

Fase 8 (implementada)

Configuración

Jobs automáticos

API

Backfill

Despliegue en VPS (Linux / HestiaCP)

Estructura

Lo que este sistema NO hace

1. Arquitectura

2. Esquema de datos

Inmutabilidad de Prediction

3. El modelo

3.1 Verosimilitud

3.2 Gradiente analítico

3.3 Matriz de marcadores

3.4 Prior de recién ascendido

3.5 Calibración a mercado

4. Fase 8

4.1 Simulación minuto a minuto (model/inplay.py) — matriz principal

4.2 Sobredispersión (model/dispersion.py)

4.3 xG (ingest/xg.py)

4.4 ξ por liga (engine/tuning.py)

4.5 Ataque/defensa casa-fuera (model/dixon-coles.py)

4.6 Kelly fraccional (engine/kelly.py)

5. Anti-fuga temporal

6. Rendimiento medido

7. Decisiones de diseño y por qué

8. Errores conocidos y cómo se manejan

1. Entorno

2. Mapa del código

3. Convenciones

4. Tocar el modelo sin romperlo

5. Tests

6. Migraciones de esquema

7. Flujo de trabajo típico

Añadir un mercado nuevo (ejemplo: «gana alguna parte»)

Añadir una liga

8. Logging y diagnóstico

9. Antes de dar algo por terminado

A. Las cinco que de verdad mueven la aguja

A1. Extraer las lambdas del mercado desde hándicap asiático y línea de goles ★ la mejor

A2. Identidad de equipo entre divisiones (ascensos y descensos)

A3. ✓ HECHO — Simulación minuto a minuto como matriz principal

A4. ✓ HECHO — ξ ajustado por liga con validación temporal

A5. ✓ HECHO — Ataque y defensa separados casa/fuera

B. Mejoras de datos (sin datos nuevos no hay milagros)

B1. Alineaciones, bajas y sanciones

B2. Congestión de calendario y competición europea

B3. xG con una fuente real

B4. Guardar cuotas en varios momentos, no solo una

C. Cómo se miden y se presentan los resultados posibles

C1. Medir con RPS, no solo con Brier y acierto top-1

C2. Calibración post-hoc (isotónica) sobre el histórico de evaluaciones

C3. Mostrar la incertidumbre de la propia predicción

C4. Agrupar los resultados posibles en escenarios

C5. Ensemble en vez de un solo modelo

D. Optimizaciones (rendimiento y coste)

D1. Caché de matrices por lambdas redondeadas

D2. Vectorizar el backtest por jornada

D3. Adelgazar el JSON de las predicciones

D4. Simulaciones adaptativas

D5. Reutilizar el ModelRun en el backfill

E. Lo que NO merece la pena (y por qué)

Plan sugerido, en orden

Para usar la aplicación. No hace falta saber programar.

1. Ponerla en marcha (una sola vez)

1. Instala **Python 3.11 o superior** desde <https://www.python.org/downloads/>. En la primera pantalla del instalador, marca «**Add python.exe to PATH**».
2. Doble clic en **instalar.bat** . Tarda un par de minutos.
3. Doble clic en **primera_carga.bat** . Descarga seis temporadas de las cuatro ligas y entrena el modelo. Menos de un minuto.
4. Doble clic en **arrancar.bat** . Se abre el navegador en <http://localhost:8017>.

A partir de ahí, para usarla solo hace falta **arrancar.bat** .

Para cerrarla: cierra la ventana negra de comandos.

2. La pantalla principal: Semana

Es lo primero que ves. Muestra los partidos de **lunes a domingo**.

Los que ya se han jugado salen con:

- El marcador final en grande, y entre paréntesis el del descanso y el de la segunda parte.
- Una etiqueta verde **1X2 OK** o roja **1X2 fallo** según si el resultado más probable del modelo fue el que salió.
- Una etiqueta **MARCADOR EXACTO** si además clavó el resultado.
- Debajo, en gris: qué probabilidad le había dado el modelo al resultado real.

Los que quedan por jugar salen con:

- Una **barra de colores** con las tres probabilidades: azul = gana el local, gris = empate, naranja = gana el visitante. El número dentro es el porcentaje.
- **Resultados posibles**: los tres marcadores más probables del partido completo (FT), los dos más probables de la primera parte (1T) y los dos de la segunda (2T), cada uno con su probabilidad.
- La probabilidad de que haya **más de 2,5 goles** y de que **marquen los dos** equipos (BTTS).
- La **cuota justa** del modelo frente a la **cuota real** de las casas.
- Una etiqueta morada **VALUE** si el modelo cree que la cuota que ofrece el mercado es demasiado alta para esa probabilidad (léete el apartado 7 antes de hacerte ilusiones).

Arriba tienes el selector de liga y los botones para moverte a la **semana anterior** o la **siguiente**.

La última columna: 1ª y 2ª parte reales

En los partidos ya jugados, la última columna muestra el desglose real:

```

1T 1-0 nº 2    <- el 1-0 era el 2º marcador más probable de la 1ª parte
2T 0-1 nº 3
FT 1-1 nº 1    <- el 1-1 era el marcador final más probable

```

Ese número es **el puesto que ocupaba el resultado real en el ranking del modelo**. Verde = era el primero, naranja = estaba entre los tres primeros, gris = ni siquiera entraba en los diez. Es la forma directa de ver si la simulación está funcionando: no hay que acertar siempre el número uno, hay que tener el resultado real cerca de la cabeza de forma habitual.

Los partidos de ayer aparecen sin resultado

Es normal. **football-data.co.uk publica sus ficheros cada pocos días, no en directo**. Un partido jugado ayer puede tardar entre 24 y 72 horas en aparecer con su marcador. Al principio de temporada, además, algunas ligas todavía no tienen fichero publicado (la Premier suele tardar varias semanas).

La aplicación te lo dice arriba, en un aviso amarillo, con la última fecha publicada de cada liga. Si ves ese aviso, no hay nada que arreglar: hay que esperar a la fuente.

La etiqueta «post-inicio»

Algunos partidos aparecen sin resultado porque la fuente de datos tarda unas horas en publicarlo. La aplicación te enseña la predicción igualmente, pero marcada **post-inicio**: se generó cuando el partido ya había empezado, así que **no cuenta** para las estadísticas de rendimiento. Es transparencia, no un error.

3. Los botones de arriba

Botón	Qué hace	Cuándo usarlo
Actualizar todo	Descarga resultados nuevos, reentrena, predice y evalúa	Es el único que necesitas normalmente. Una vez al día
Ingesta	Solo descarga datos nuevos	Si sabes que acaba de terminar una jornada
Entrenar	Solo recalcula las fuerzas de los equipos	Tras cambiar algo en la configuración
Predecir	Solo genera predicciones de los próximos partidos	Si has añadido partidos nuevos
Evaluar	Puntúa los partidos ya jugados que tenían predicción	Después de una jornada

Si tienes la aplicación abierta, esto se hace **solo**: a las 7:00 descarga datos, a las 7:30 y 13:00 predice, y a las 23:30 evalúa.

4. Ficha de un partido

Haz clic en el nombre de cualquier partido.

Lo primero: el desglose por tiempos

Haz clic en cualquier marcador final —tanto en la tabla «Marcador exacto» como en una casilla del mapa de calor— y a la derecha se despliega **con qué primera y segunda parte se llega a ese resultado**, ordenado de más a menos frecuente.

Por ejemplo, para un 1-1 puede salir:

1ª parte	2ª parte	Frecuencia
0-0	1-1	32 %
1-0	0-1	25 %
0-1	1-0	24 %
1-1	0-0	19 %

Esos porcentajes **suman 100 % dentro del 1-1**: no son la probabilidad del partido, son la probabilidad del camino dado que acaba 1-1. Además, el reparto más probable se marca en morado sobre las matrices de cada tiempo.

Si el partido ya se jugó, se selecciona automáticamente el resultado real y el camino que ocurrió de verdad sale marcado en verde con la etiqueta **REAL**.

Las ayudas (?)

Cada apartado tiene un icono (?) al lado del título. Pasa el ratón por encima (o púlsalo) y sale una explicación de qué es ese bloque y cómo leerlo. Están en todas las páginas.

Los colores

- **Azul intenso**: el resultado más probable.
- **Azul medio y suave**: el segundo y el tercero.
- **Gris atenuado**: la cola, poco probable.
- **Verde**: el resultado REAL del partido.
- **Morado**: lo que has seleccionado.
- **Mapa de calor de marcadores**. Una cuadrícula: las filas son los goles del local, las columnas los del visitante, y el color indica lo probable que es cada marcador. Hay uno del partido completo y otros dos por mitades. Si el partido ya se jugó, la casilla del resultado real sale con **borde verde**.
- **Marcador exacto**: los diez más probables de cada tiempo, con su cuota justa.
- **Todos los mercados**: 1X2, doble oportunidad, más/menos goles de 0,5 a 4,5, ambos marcan, hándicap y las nueve combinaciones descanso/final.

- **Simulación minuto a minuto:** la aplicación simula 20.000 veces el partido teniendo en cuenta que se marcan más goles al final y que el equipo que va perdiendo ataca más. De ahí salen las probabilidades de **remontada** y los goles esperados por tramos de 15 minutos.
 - **Contraste con la realidad** (solo en partidos jugados): una tabla con el marcador real de cada tiempo, en qué puesto del ranking estaba y qué probabilidad le dio el modelo. Es la comprobación honesta de si la simulación funciona.
 - **Cuotas guardadas** de todas las casas disponibles.
 - **Histórico de predicciones:** cada vez que se predice ese partido se guarda una fila nueva. Las anteriores **nunca** se borran ni se modifican. Es la prueba de que las predicciones se hicieron antes y no se han retocado después.
-

5. Rendimiento

Aquí está la parte incómoda y la más importante.

- **Brier del modelo** frente a **Brier del mercado**. El Brier mide cuánto se equivocan las probabilidades: **cuanto más bajo, mejor**. La referencia es el mercado (las cuotas de cierre de Pinnacle), que es muy difícil de batir.
- **Delta:** la diferencia. Si es negativa, el modelo va por delante del mercado. Si es positiva, por detrás. Lo normal es que sea positiva.
- **Curva de calibración:** cada punto es un grupo de predicciones. Si el modelo dice «40 %» y esas cosas pasan el 40 % de las veces, el punto cae sobre la diagonal. Cuanto más pegados a la diagonal, mejor calibrado.
- **ROI de tres estrategias** simuladas con 1 unidad por apuesta:
 - **(a)** apostar siempre al resultado más probable del modelo,
 - **(b)** apostar solo cuando hay value,
 - **(c)** apostar siempre al marcador exacto más probable.

La (c) está ahí a propósito, para que veas con números que apostar al marcador exacto pierde dinero de forma sistemática.

Aviso de ruido: por debajo de 200 partidos evaluados la aplicación te lo dice en grande. Con pocos partidos, cualquier número de esta página es casualidad.

6. Backtest

Simula qué habría pasado si hubieras usado el sistema en el pasado. Entrena con lo anterior a cada jornada, predice la siguiente, y avanza. **Nunca** usa información posterior al partido que predice.

Elige liga y número de temporadas y pulsa **Ejecutar backtest**. Tarda entre 10 segundos y 2 minutos.

Si el resultado sale **mucho mejor** que el mercado, la propia aplicación te avisa de que probablemente haya un error, no un hallazgo. Batir al mercado de forma consistente es rarísimo.

7. Staking (Kelly) — léelo entero

Esta página calcula cuánto arriesgaría el criterio de Kelly en las selecciones marcadas con VALUE.

Kelly completo maximiza el crecimiento del dinero **solo si las probabilidades son exactas**. Las de este sistema no lo son: la página de Rendimiento te lo está enseñando. Por eso:

- Se usa **Kelly 1/4** por defecto.
- Hay un **tope duro del 2 %** del banco por apuesta.
- Se descarta cualquier ventaja inferior al 3 %.

Esto no es una recomendación de apuesta. Es el cálculo de cuánto arriesgaría un criterio matemático con unas probabilidades que sabemos imperfectas. Mientras la estrategia (b) de la página de Rendimiento no dé ROI positivo con más de 300 apuestas, este plan es un ejercicio.

8. Modelo

Ver la «fuerza» de cada equipo según el sistema:

- **Ataque:** cuántos goles marca de más (positivo) o de menos (negativo) que la media de su liga.
- **Defensa:** cuántos encaja de menos (positivo) o de más (negativo).
- **Rating:** la suma. Ordena la tabla.
- **Fortín:** cuánto mejor es ese equipo concreto en casa que fuera, por encima de la ventaja local general de la liga. Positivo = se crece en su campo.
- **Ventaja local:** cuánto multiplica jugar en casa los goles esperados.
- **Ajustar ξ:** el botón lanza un barrido que busca, para esa liga, cuánta memoria conviene tener (si pesa más lo reciente o lo de hace dos años). Tarda unos 30 segundos por liga y el valor elegido queda guardado.
- Etiqueta **prior** en equipos recién ascendidos o con muy pocos partidos: el sistema no se fía de ellos y los trata con prudencia.

Abajo tienes las **20 ligas** disponibles. Vienen activas Premier, Championship, LaLiga, Hypermotion, Serie A, Ligue 1 y Primeira Liga. Pulsa en el estado para activar o desactivar cualquier otra. Después, **Ingesta histórica completa** y **Reentrenar**.

9. Preguntas frecuentes

¿De dónde salen los datos? De football-data.co.uk, que publica CSV gratuitos con resultados, resultados al descanso y cuotas de varias casas. Es la única fuente. No se scrapea nada.

¿Por qué un partido no tiene predicción? Porque cuando se ejecutó el ciclo de predicción ese partido no estaba todavía en `fixtures.csv`, o porque su liga no está activa. Pulsa **Actualizar todo**.

¿Por qué faltan los resultados de ayer? Porque la fuente todavía no los ha publicado. Mira el apartado 2: los CSV de football-data.co.uk se actualizan cada pocos días. El aviso amarillo del dashboard te dice hasta qué fecha llega cada liga.

¿Por qué a veces sale «sin cuotas»? Porque la fuente no publicó cuotas para ese partido. La predicción sigue siendo válida, pero sin cuotas no se puede calcular value ni ROI.

¿Puedo cambiar las ligas o el número de días que predice? Sí, en `config.yaml`. Las ligas también desde la página Modelo.

¿Los datos se me borran si cierro el programa? No. Todo está en `data\quiniela.db`. Haz copia de ese fichero si quieres guardarlo.

¿Puedo usarlo desde el móvil? Solo dentro de tu red, y no viene configurado para eso. La versión para servidor está documentada en el README.

¿El sistema gana dinero? No. Y está construido precisamente para enseñarte por qué. Las tres estrategias simuladas suelen dar ROI negativo, el modelo va por detrás del mercado, y la propia aplicación te avisa cuando un resultado es demasiado bueno para ser cierto.

Herramienta de análisis estadístico. Las probabilidades no garantizan resultados. Si apuestas: solo dinero que puedas perder, y estas simulaciones muestran que batir al mercado de forma sostenida es muy improbable. Juego responsable: jugarbien.es — Tel. 900 200 225 (España).

Quiniela Engine

Sistema de predicción y evaluación de resultados de fútbol. Lee los partidos de la semana, muestra **los resultados reales** de los que ya se han jugado y **los resultados posibles** (con probabilidad y cuota justa) de los que quedan.

Modelo Dixon-Coles con decaimiento temporal ajustado por liga, ataque/defensa separados casa/fuera y **simulación minuto a minuto como motor principal**. FastAPI + SQLite, sin build de frontend, sin broker, sin Docker.

```
python run.py --setup -> http://localhost:8017
```

Instalación en Windows (3 pasos)

1. `instalar.bat` — crea `.venv` e instala dependencias (necesita Python 3.11+ en el PATH).
2. `primera_carga.bat` — descarga 6 temporadas de las 7 ligas activas, entrena y genera las primeras predicciones. Tarda menos de un minuto con conexión normal.
3. `arrancar.bat` — levanta el servidor y abre el navegador.

Sin los `.bat` :

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe run.py --setup
```

Otros comandos:

Comando	Qué hace
<code>python run.py</code>	Arranca (usa los datos ya descargados)
<code>python run.py --setup</code>	Ingesta + entrenamiento + predicción y arranca
<code>python run.py --no-scheduler</code>	Sin jobs periódicos
<code>python run.py --port 8080</code>	Otro puerto
<code>python -m app.engine.backtest --league SP1 --seasons 4</code>	Backtest por consola
<code>pytest</code>	Batería de tests (165)

Qué ves en la aplicación

Página	Contenido
Semana /	Los partidos de lunes a domingo. Los jugados con su marcador final y una última columna con la 1ª y la 2ª parte reales y el puesto que ocupaba cada uno en el ranking del modelo; los pendientes con probabilidades 1X2, los 3 marcadores más probables de cada tiempo, cuota justa contra cuota de mercado y badge VALUE. Navegación por semanas, avisos de frescura de datos y ayudas flotantes (?) en cada columna.
Próximos /proximos	La ventana completa de predicción (8 días por defecto), en tabla densa. Filtro «solo con value».
Partido /match/{id}	Heatmap de la matriz de marcadores (partido completo y por mitades), todos los mercados derivados, la simulación minuto a minuto y el histórico de predicciones. Haz clic en cualquier marcador final (en la tabla o en la matriz) y se despliega con qué primera y segunda parte se llega a él, ordenado por frecuencia. Si el partido ya se jugó, el resultado real sale marcado en verde en todas las tablas y hay un bloque de contraste que dice en qué puesto del ranking estaba. Cada apartado tiene una ayuda flotante (?).
Rendimiento /stats	Brier del modelo contra Brier del mercado, curva de calibración, ROI acumulado de tres estrategias y tabla por liga. Avisa en grande si hay menos de 200 partidos evaluados.
Backtest /backtest	Walk-forward configurable con gráfico de bankroll.
Staking /kelly	Kelly fraccional sobre las selecciones con value, exportable a CSV.
Modelo /modelo	Fuerzas de ataque/defensa por equipo, columna fortín (cuánto mejor es cada equipo en casa), parámetros del ajuste, botón de ajuste de ξ y activación de las 20 ligas.

Todas las páginas llevan el aviso fijo de juego responsable en el pie.

Cómo funciona

Datos

Solo CSV públicos de football-data.co.uk: histórico por temporada (`mmz4281/{temporada}/{liga}.csv`) y próximos partidos (`fixtures.csv`). No se scrapea nada. La ingesta es idempotente: cada fila se identifica por su clave natural y por un hash, así que volver a ingestar no duplica ni pisa nada.

Ligas activas por defecto: **E0** (Premier), **E1** (Championship), **SP1** (LaLiga), **SP2** (Hypermotion), **I1** (Serie A), **F1** (Ligue 1) y **P1** (Primeira Liga). Hay 20 en el catálogo; se activan desde `/modelo` o desde `config.yaml`.

Sobre la frescura de los datos: football-data.co.uk publica sus CSV **cada pocos días, no en directo**. Un partido jugado ayer puede tardar 24-72 h en aparecer con resultado, y al principio de temporada

algunas ligas todavía no tienen fichero publicado. El dashboard lo avisa de forma explícita (GET /api/data-freshness) para no confundirlo con un fallo del sistema.

Modelo

```
log λ_local      = μ + ataque_local - defensa_visitante + γ
log λ_visitante = μ + ataque_visit. - defensa_local
```

- Corrección **τ de Dixon-Coles** para 0-0, 1-0, 0-1 y 1-1.
- Peso temporal $w(t) = \exp(-\xi \cdot \text{días}/365)$, con ξ ajustado por liga (ver abajo).
- Restricciones $\Sigma \text{ataque} = 0$, $\Sigma \text{defensa} = 0$; μ es el nivel base de goles de la liga (el SPEC lo omite en la fórmula; aquí es explícito porque sin él las restricciones forzarían un nivel medio de 1 gol — es el mismo modelo).
- Ajuste por **L-BFGS-B con gradiente analítico**: LaLiga con 6 temporadas entrena en menos de un segundo.
- **ξ ajustado por liga** (fase A4): cada competición tiene su propia memoria. Medido: Serie A 0.20, Ligue 1 0.35, LaLiga 0.50, Premier 1.00, Championship 1.30 — frente al 1.5 único de antes.
- **Ataque y defensa desdoblados casa/fuera** (fase A5), con penalización L2 de peso 40. Sin esa regularización el desdoble empeora el modelo: se pasa de 2 a 4 parámetros por equipo con los mismos datos.
- Equipos con menos de 6 partidos en la ventana reciben prior de recién ascendido (percentil 20 de ataque y de fuerza defensiva) con shrinkage lineal.
- **Calibración a mercado** opcional (activada): media geométrica ponderada entre el modelo y las probabilidades implícitas sin margen, peso de mercado 0.7. Se guardan **siempre las dos** (modelo puro y blend) para poder puntuar ambas.

Tiempos

La fracción de goles en la primera parte se estima del histórico de la liga (≈ 0.43 en LaLiga). Cada mitad es una Poisson con la misma p .

Las matrices que ve el usuario **no** salen de este supuesto: la matriz principal es la de la simulación minuto a minuto, que sí modela la dependencia entre mitades. Las de Dixon-Coles se siguen calculando como referencia y respaldo, y la ficha de partido enseña la diferencia entre ambas.

Evaluación

Al terminar un partido con predicción previa se calcula Brier, log-loss, aciertos de marcador exacto (FT / 1ª / 2ª parte), la probabilidad que se asignó al resultado real, CLV contra la cuota de cierre y el ROI de tres estrategias con stake 1:

- **(a)** siempre al 1X2 más probable del modelo,
- **(b)** solo cuando $p \cdot \text{cuota} > 1.05$,
- **(c)** siempre al marcador exacto top-1 — está ahí precisamente para enseñar su ROI negativo (cuota sintética = justa / 1.30, porque los CSV no traen cuotas de marcador exacto).

Criterio de modelo sano: Brier del modelo $\leq$ Brier del mercado + 0.005 con $\geq$ 200 partidos. Con los datos reales de 2024-2026 el resultado típico es delta $\approx$ +0.017: **el modelo va por detrás del mercado**, que es lo esperable y lo honesto. Si alguna vez sale muy por delante, la app lo dice: casi siempre significa fuga de datos.

Backtest

Walk-forward estricto: para predecir el día D solo entra histórico con fecha **estrictamente anterior** a D. Lo garantiza `predictor.training_matches` y lo verifica `tests/test_backtest_leak.py` , que espía cada ajuste durante un backtest real.

Fase 8 (implementada)

Mejora	Dónde	Qué hace
Simulación minuto a minuto (A3)	<code>app/model/inplay.py</code>	Monte Carlo de 20.000 partidos con tasa de gol dependiente del minuto y del marcador . Es la matriz principal del sistema . Su intensidad de dependencia se calibra contra la distribución HT/FT real de cada liga (en LaLiga sale 0.5: la mitad de fuerte que los valores de manual). Guarda además el reparto conjunto (descanso / final) que alimenta el desglose por tiempos. Reproducible por semilla.
ξ por liga (A4)	<code>app/engine/tuning.py</code>	Barrido de ξ con el backtest walk-forward, eligiendo por log-loss. Guarda el ganador en la base de datos. <code>POST /api/admin/tune-ξ</code> .
Casa/fuera desdoblado (A5)	<code>app/model/dixon_coles.py</code>	<code>a ± δa</code> , <code>d ± δd</code> con penalización L2. Gradiente analítico verificado contra diferencias finitas. Columna «fortín» en <code>/modelo</code> .
Sobredispersión	<code>app/model/dispersion.py</code>	Binomial negativa opcional (<code>model.distribution: negbin</code>). k se estima por momentos sobre los residuos del ajuste. Más cola: más 0-0 y más goleadas de las que predice Poisson.
xG externo	<code>app/ingest/xg.py</code>	Importa xG de un CSV local (<code>data/xg/SP1.csv</code>) o de una URL propia y mezcla las fuerzas resultantes con las del modelo (peso 0.35). No scrapea nada : los datos los pones tú.
Ligas adicionales	<code>config.yaml</code> + <code>/modelo</code>	Catálogo de 20 ligas, activables con un clic.
Kelly fraccional	<code>app/engine/kelly.py</code> + <code>/kelly</code>	Kelly 1/4 por defecto, tope duro del 2 % del banco por apuesta, edge mínimo del 3 %, exportación a CSV.

Configuración

Todo en `config.yaml`. Lo que más se toca:

```
model:
  xi: 1.5                # decaimiento por defecto (se ajusta por liga)
  calibrate_to_market: true # mezcla con el mercado
  market_weight: 0.7
  distribution: "poisson"  # o "negbin"
  primary_matrix: "simulacion" # o "poisson"
  calibrate_state: true    # ancla la dependencia al HT/FT real de la liga
  home_away_split: true
  home_away_reg: 40.0     # NO bajarlo sin comprobarlo con backtest
predict:
  horizon_days: 8
  include_past_days: 2    # fixtures ya empezados sin resultado publicado
value:
  threshold: 1.05
inplay:
  enabled: true
  n_sims: 20000
kelly:
  fraction: 0.25
```

Variables de entorno que mandan sobre el fichero: `QE_DB_URL`, `QE_HOST`, `QE_PORT`, `QE_SCHEDULER`, `ADMIN_KEY`, `QE_CONFIG`.

Jobs automáticos

Job	Hora	Qué hace
<code>daily_ingest</code>	07:00	Temporada en curso + fixtures
<code>predict_cycle</code>	07:30 y 13:00	Reentrena si hace falta y predice
<code>evaluate_cycle</code>	23:30	Evalúa lo que ya se ha jugado

Todos se pueden lanzar a mano desde los botones del dashboard o desde `/api/admin/*`.

API

Lectura (abierta):

```

GET /api/leagues
GET /api/matches?league=&status=
GET /api/predictions/upcoming
GET /api/stats/summary
GET /api/stats/timeline
GET /api/kelly
GET /api/data-freshness

GET /api/matches/week?league=&offset=
GET /api/matches/{id}
GET /api/model/{liga}
GET /api/stats/calibration
GET /api/stats/by-league
GET /api/kelly/export.csv
GET /api/health

```

Administración (cabecera `X-Admin-Key` si `ADMIN_KEY` está definida):

```

POST /api/admin/ingest
POST /api/admin/train
POST /api/admin/evaluate
POST /api/admin/backfill
POST /api/admin/xg/import
POST /api/admin/tune-xi

POST /api/admin/ingest/refresh
POST /api/admin/predict
POST /api/admin/backtest
POST /api/admin/full-cycle
POST /api/admin/leagues/{code}/toggle

```

Documentación interactiva en `/docs`.

Backfill

`POST /api/admin/backfill` genera predicciones **retrospectivas sin fuga temporal** (entrena con `as_of` = fecha del partido) para que la página de Rendimiento tenga datos desde el primer día. Van marcadas con `params.backfill = true`.

Despliegue en VPS (Linux / HestiaCP)

```
sudo bash deploy/instalar_vps.sh
```

Después: cambia `ADMIN_KEY` en `/etc/systemd/system/quiniela.service`, reinicia el servicio y monta el proxy inverso con `deploy/nginx-hestia.conf` (puerto interno 8017, timeouts de 600 s para el backtest, `/api/admin/` restringido por IP o basic auth).

Misma base de código que en Windows. Para migrar a PostgreSQL basta con cambiar `QE_DB_URL`; el ORM no usa nada específico de SQLite.

Estructura

```

quiniela-engine/
├─ run.py                entry point (uvicorn + scheduler)
├─ config.yaml           ligas, parámetros del modelo, jobs
├─ app/
│  ├─ main.py            db.py  models.py  config.py  scheduler.py
│  ├─ ingest/            football_data.py  mapper.py  xg.py
│  ├─ model/             dixon_coles.py  halves.py  markets.py  inplay.py  dispersion.py
│  ├─ engine/            predictor.py  evaluator.py  value.py  backtest.py  kelly.py
│  │                       tuning.py
│  ├─ api/               routes_data.py  routes_admin.py  schemas.py
│  └─ web/               templates/  static/
├─ tests/                165 tests
├─ deploy/               systemd + nginx + instalador VPS
└─ docs/                 documentación técnica, de desarrollo y de usuario

```

Lo que este sistema NO hace

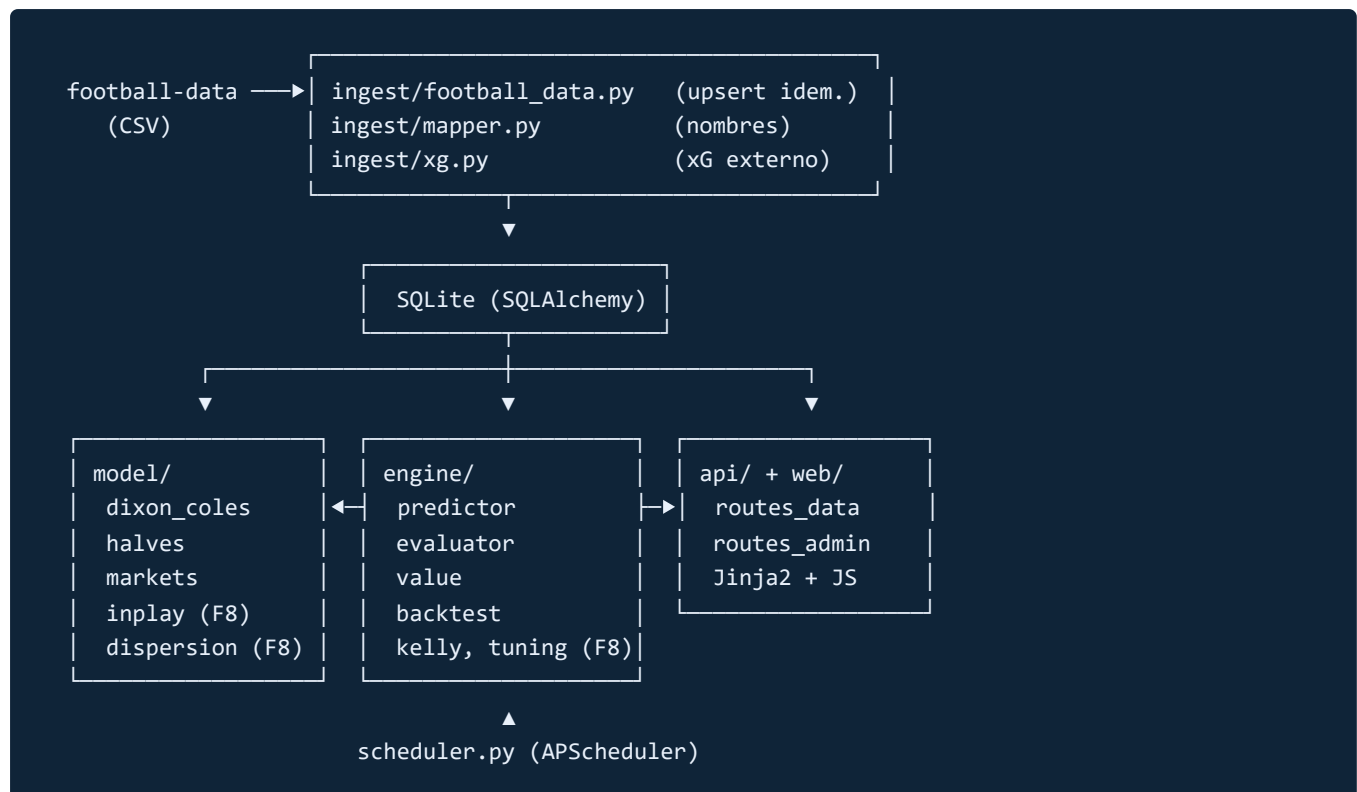
- No se conecta a casas de apuestas ni apuesta por su cuenta.
- No scrapea sitios que lo prohíban.
- **No promete rentabilidad.** La propia página de Rendimiento está diseñada para enseñarte que batir al mercado de forma sostenida es muy improbable, y las estrategias simuladas suelen salir en negativo. Es una herramienta de análisis y de aprendizaje estadístico.

Herramienta de análisis estadístico. Si apuestas: solo dinero que puedas perder. Juego responsable: jugarbien.es
 — Tel. 900 200 225 (España).

Documentación técnica — Quiniela Engine

Versión 1.0.0 · Python 3.11+ · FastAPI + SQLAlchemy 2 + SQLite

1. Arquitectura



Sin capas innecesarias: los módulos de `model/` son funciones puras sobre `numpy`, los de `engine/` orquestan y tocan la base de datos, y `api/` solo serializa. Esa separación es lo que permite que `backtest.py` reutilice el mismo código de predicción sin escribir nada en la base.

2. Esquema de datos

Tabla	Claves	Notas
league	code único	20 ligas sembradas desde <code>config.yaml</code> en cada arranque
team	(league_id , canonical_name) único	canonical_name = nombre normalizado (minúsculas, sin acentos ni apóstrofes)
match	(league_id , season , date , home_id , away_id) único	source_hash = SHA1 de los campos relevantes del CSV → idempotencia
odds_snapshot	(match_id , bookmaker , is_closing) único	Se guardan los 8 bloques disponibles (B365, Pinnacle, Average, Max × pre/cierre)
model_run	índice en trained_at	params_json = DCPParams serializado completo
prediction	índice (match_id , created_at)	Immutable
evaluation	prediction_id único	1:1 con la predicción evaluada
match_xg	(match_id , source) único	Fase 8, opcional

Índices explícitos: `match(league_id,date)` , `match(status)` , `prediction(match_id,created_at)` , `odds_snapshot(match_id,is_closing)` .

SQLite se abre con `journal_mode=WAL` , `synchronous=NORMAL` y `foreign_keys=ON` . Nada del ORM es específico de SQLite: cambiando `QE_DB_URL` a `postgresql+psycopg://...` funciona igual.

Inmutabilidad de Prediction

Una predicción **nunca** se actualiza. Repredcir crea una fila nueva. Lo garantiza el diseño de `predictor.store_prediction` (solo `session.add`) y lo verifica `tests/test_predictor.py::test_prediccion_no_se_modifica_al_repredcir` , que compara un snapshot campo a campo tras forzar una repredicción.

Para evaluar se usan dos selectores distintos:

- `official_prediction(session, match)` — la que se **muestra**: la última creada antes del saque inicial; si no hay ninguna, la más reciente.
- `evaluable_prediction(session, match)` — la que **cuenta**: solo si es anterior al saque inicial y no está marcada `post_kickoff` .

3. El modelo

3.1 Verosimilitud

Para cada partido m con peso $w_m = \exp(-\xi \cdot \Delta \text{días}/365)$:

$$\log L = \sum w_m [\log \tau(x,y) - \lambda + x \cdot \log \lambda - \mu + y \cdot \log \mu]$$

con $\log \lambda = \mu_0 + a_h - d_a + \gamma$ y $\log \mu = \mu_0 + a_a - d_h$, y

$$\begin{aligned} \tau(0,0) &= 1 - \lambda\mu\rho & \tau(1,0) &= 1 + \mu\rho \\ \tau(0,1) &= 1 + \lambda\rho & \tau(1,1) &= 1 - \rho & \tau &= 1 \text{ en el resto} \end{aligned}$$

Los factoriales se omiten: son constantes respecto a los parámetros.

3.2 Gradiente analítico

Es la razón de que el ajuste tarde milisegundos en lugar de decenas de segundos. Con $g_\lambda = w[(x - \lambda) + (\partial\tau/\partial\log\lambda)/\tau]$ y su simétrico g_μ :

$$\begin{aligned} \partial/\partial\mu_0 &= \sum (g_\lambda + g_\mu) \\ \partial/\partial\gamma &= \sum g_\lambda \\ \partial/\partial a_i &= \sum_{\{local=i\}} g_\lambda + \sum_{\{visit=i\}} g_\mu & (\text{np.bincount}) \\ \partial/\partial d_i &= -\sum_{\{visit=i\}} g_\lambda - \sum_{\{local=i\}} g_\mu \\ \partial/\partial\rho &= \sum w (\partial\tau/\partial\rho)/\tau \end{aligned}$$

Identificabilidad por penalización cuadrática $100 \cdot [(\sum a)^2 + (\sum d)^2]$ durante la optimización y recentrado exacto al terminar (la media se traslada a μ_0 , así el nivel de goles de la liga no cambia).

Guardas numéricas: $\log \lambda$ recortado a $[-6, 2.5]$, τ con suelo $1e-8$ (y gradiente anulado donde se recorta), ρ acotada a $[-0.25, 0.25]$, λ finales a $[0.02, 8]$.

3.3 Matriz de marcadores

`score_matrix( $\lambda$ ,  $\mu$ ,  $\rho$ ,  $n$ , distribution,  $k$ )` construye el producto exterior de las dos marginales, aplica τ a las cuatro celdas bajas y **renormaliza** para que sume exactamente 1 (la cola por encima de $n-1$ se reparte proporcionalmente). Test: `test_matriz_suma_uno`, `test_matriz_sin_rho_es_producto_de_poissons`, `test_tau_se_aplica_solo_a_los_cuatro_marcadores_bajos`.

3.4 Prior de recién ascendido

Equipos con `< min_matches_new_team` (6) partidos en la ventana reciben el percentil 20 de ataque y de **fuerza defensiva** de los equipos consolidados, con shrinkage lineal $n/6$ hacia su propia estimación.

Nota de signo: en esta parametrización d_i es **fuerza** defensiva (entra restando, más alto = encaja menos). El percentil 20 de fuerza equivale al percentil 80 de goles encajados que pide el SPEC.

3.5 Calibración a mercado

$$p_{\text{blend}} \propto p_{\text{modelo}}^{(1-w)} \cdot p_{\text{mercado}}^w \quad (w = 0.7)$$

Se guardan `p_model`, `p_market` y `p_blend` en `Prediction.params_json`. El evaluador puntúa las tres series por separado (`brier_1x2`, `brier_blend`, `brier_market`), así se ve cuánto aporta cada parte.

4. Fase 8

4.1 Simulación minuto a minuto (`model/inplay.py`) — matriz principal

$$p_{\text{gol_local}}(t) = (\lambda_{\text{local}}/90) \cdot \text{perfil}(t) \cdot \text{estado}(\text{marcador})$$

- `perfil(t)` sube linealmente de 0.78 a 1.22, normalizado a media 1.
- `estado`: $\times 1.16 / \times 1.28 / \times 1.34$ para quien pierde por 1 / 2 / 3+; $\times 0.93 / \times 0.87 / \times 0.84$ para quien gana.

Bucle de 90 iteraciones vectorizado sobre las N simulaciones (numpy), no un bucle por simulación: 20.000 partidos en ~70 ms. Semilla fija $\Rightarrow$ predicciones reproducibles.

Devuelve matrices FT/1T/2T, las 9 combinaciones HT/FT, probabilidades de remontada y goles esperados por tramos de 15 minutos. **Aquí las mitades no son independientes**, que es justo lo que el modelo Poisson no puede dar.

Los goles esperados se conservan dentro de un 12 % (test `test_simulacion_conserva_los_goles_esperados`): la simulación redistribuye, no inventa goles.

Calibración de la dependencia. Los multiplicadores de estado son valores de manual. Para anclarlos a la realidad haría falta el minuto de cada gol, que football-data no publica. La solución que usa el sistema: `calibrate_state_strength` introduce un escalar `strength` que multiplica la desviación respecto a 1 (0 = independencia total, 1 = valores de manual) y barre una rejilla eligiendo el valor cuya distribución HT/FT simulada minimiza la distancia L1 contra la **distribución HT/FT observada en el histórico de la liga**. Nueve celdas para estimar un parámetro: sobra información.

Medido en LaLiga: `strength  $\approx$  0.5`. La dependencia real es **la mitad de fuerte** que los valores de manual; sin calibrar, el modelo la exageraba. La calibración se hace una vez por entrenamiento y se guarda en `DCParams.state_strength`.

Reparto conjunto (`_joint_decomposition`). Además de las marginales se cuenta el tensor (`marcador final  $\times$  marcador al descanso`) con `np.bincount` sobre un índice aplanado ($8 \cdot 8 \cdot 6 \cdot 6 = 2304$ celdas). De ahí sale, para cada marcador final, la distribución **condicionada** de cómo se llegó a él. Es lo que alimenta el desglose 1T/2T de la ficha de partido, y es información que dos Poisson independientes no pueden dar: refleja que un equipo que se pone por delante baja el ritmo.

Existe también la versión analítica para el caso independiente (`halves.decompose_ft`), que se usa cuando `primary_matrix: "poisson"`.

Promoción a matriz principal. Con `model.primary_matrix: "simulacion"` las matrices FT/1T/2T que consumen todos los mercados salen de la simulación. Las de Dixon-Coles se siguen calculando siempre — son la referencia y el respaldo — y la ficha de partido muestra la diferencia entre ambas en el bloque «comparativa». El backtest acepta `simulate=True/False` para medir cuál calibra mejor (`--no-sim` en la CLI); con `BACKTEST_SIMS = 4000` en vez de 20.000, porque ahí importa la media sobre miles de partidos, no la cola de cada uno.

4.2 Sobredispersión (`model/dispersion.py`)

Binomial negativa con $\text{Var} = \lambda + \lambda^2/k$, parametrizada como `nbinom(r=k, p=k/(k+λ))`. k se estima por el método de los momentos sobre los residuos del ajuste Dixon-Coles, así el modelo de fuerzas no cambia: solo cambia la distribución que convierte λ en matriz. $k \geq 500$ es indistinguible de Poisson.

4.3 xG (`ingest/xg.py`)

No hay scraping. Entrada: CSV local `data/xg/<LIGA>.csv` o URL declarada en `config.yaml`. Se calcula un ataque/defensa alternativo en escala log a partir del xG ponderado con el mismo decaimiento temporal, y se mezcla con las fuerzas del ajuste (`weight 0.35`), recentrando después para no mover el nivel de goles.

4.4 ξ por liga (`engine/tuning.py`)

Barrido de $\xi \in \{0.2 \dots 2.5\}$ por liga usando el backtest walk-forward, eligiendo por **log-loss** y no por Brier: el log-loss castiga la sobreconfianza, que es exactamente el síntoma de un ξ mal puesto. El ganador se guarda en `League.xi` (más `xi_tuned_at` y `xi_logloss`) y `predictor.fit_params` lo usa a partir de ese momento; si es `None` se usa el de `config.yaml`.

El barrido corre con `simulate=False` y `blend=False` a propósito:

- ξ afecta a las **fuerzas** de los equipos, no a cómo se convierte λ en matriz, así que la matriz Dixon-Coles ordena igual y tarda segundos en vez de minutos.
- con el blend activo el mercado tapa las diferencias entre ξ y el barrido no distingue nada.

Resultado medido (temporadas 2024-2026): cada liga quiere una memoria distinta, que era la hipótesis. Serie A 0.20, Ligue 1 0.35, LaLiga 0.50, Primeira 0.80, Premier 1.00, Championship 1.30.

4.5 Ataque/defensa casa-fuera (`model/dixon-coles.py`)

```
log λ_local      = μ + (a_h + δa_h) - (d_a - δd_a) + γ
log λ_visitante = μ + (a_a - δa_a) - (d_h + δd_h)
```

Parámetros: $3 + 4n$ en vez de $3 + 2n$. Penalización L2 $\text{reg} \cdot \Sigma(\delta a^2 + \delta d^2)$ con `reg = 40` por defecto. **La regularización no es opcional:** sin ella se pasa de 2 a 4 parámetros por equipo con los mismos datos y el modelo empeora. Además resuelve la degeneración entre la media de los δ y γ eligiendo la solución de norma mínima.

Gradiente analítico añadido:

$$\begin{aligned}\partial/\partial\delta a_i &= \sum_{\{local=i\}} g_\lambda - \sum_{\{visit=i\}} g_\mu \\ \partial/\partial\delta d_i &= \sum_{\{visit=i\}} g_\lambda - \sum_{\{local=i\}} g_\mu\end{aligned}$$

verificado contra diferencias finitas con error relativo máximo **2e-5** (script en `docs/DOC_DESARROLLO.md` §4). Los equipos con menos de `min_matches_new_team` partidos ven su δ anulado proporcionalmente: de un equipo con 3 partidos no se puede decir nada sobre si es mejor en casa.

`DCParams.home_boost(team) =  $\delta a + \delta d$` es la columna «fortín» de `/modelo`.

4.6 Kelly fraccional (`engine/kelly.py`)

`f = (p·b - q)/b`, luego `f·fracción` (1/4), luego tope duro `max_stake_pct` (2 %). Descarta edges por debajo del 3 %. Exporta a CSV con `;`.

5. Anti-fuga temporal

`training_matches` filtra `Match.date < as_of` — **estrictamente menor**, nunca `<=`. Un partido de la misma fecha que el predicho no puede entrar.

`tests/test_backtest_leak.py` lo verifica de tres formas:

1. Directamente sobre `training_matches` con un corte en medio del histórico.
2. Con un espía sobre `fit_params` y `training_matches` durante un backtest real: comprueba que `as_of` nunca es `None`, que los cortes avanzan en el tiempo y que `max(fecha entrenada) < as_of` en cada llamada.
3. Lo mismo sobre `backfill_league`, que es lo que puebla `/stats`.

Además, el backtest **avisa** si el modelo bate al mercado por más de 0.02 de Brier: en ese caso lo más probable es que haya una fuga, no que el modelo sea bueno.

6. Rendimiento medido

Operación	Tiempo real (CPU normal, Windows)
Ingesta 4 ligas × 6 temporadas + fixtures	28 s (criterio SPEC: < 3 min)
Entrenar SP1, 6 temporadas (1.507 partidos)	< 1 s (criterio SPEC: < 30 s)
Simulación minuto a minuto, 20.000 partidos (con reparto conjunto)	70 ms
Calibración de la dependencia (5 × 12.000 simulaciones)	0.2 s
Barrido de ξ , 9 valores × 3 temporadas, una liga	~30 s
Backtest SP1 2 temporadas (refit semanal)	~3 s
Backfill + evaluación 4 ligas, 2 temporadas (1.804 partidos)	28 s
Batería completa de tests (135)	18 s

Resultado real sobre 1.804 partidos de E0/E1/SP1/SP2 (temporadas 2025-26 y 2026-27):

```
Brier modelo    0.6265
Brier blend     0.6126
Brier mercado   0.6091    delta = +0.0174  -> modelo por detrás del mercado
acierto 1X2     49.1 %
ROI (a) favorito del modelo  -4.2 %
ROI (b) value           -14.0 %
ROI (c) marcador exacto    -29.4 %
```

Es el resultado esperado y honesto. Un delta negativo grande sería sospechoso.

7. Decisiones de diseño y por qué

Decisión	Motivo
μ_0 explícito en la fórmula	Sin él, $\Sigma a = \Sigma d = 0$ forzaría un nivel medio de 1 gol por equipo
Gradiente analítico	40+ parámetros; diferencias finitas costarían ~44 evaluaciones por gradiente
Penalización + recentrado en vez de restricciones duras	L-BFGS-B no admite restricciones de igualdad; el recentrado final es exacto
Predicciones inmutables	Sin esto no se puede demostrar que el modelo no se ajustó a posteriori
Blend guardado junto al modelo puro	Permite medir cuánto del acierto viene del mercado y cuánto del modelo
<code>include_past_days</code> + marca <code>post_kickoff</code>	El dashboard no queda con huecos, pero esas predicciones no contaminan las métricas
Backfill separado del ciclo normal	Poblar <code>/stats</code> desde el primer día sin mentir sobre cuándo se predijo
Simulación con semilla fija	Dos ejecuciones de la misma predicción dan lo mismo; si no, no sería auditable
Sin Composer/npm/Docker	Un <code>pip install</code> y un <code>python run.py</code> en Windows

8. Errores conocidos y cómo se manejan

Situación	Comportamiento
CSV de temporada que aún no existe (HTTP 300/404)	Se registra como «no disponible» y se sigue. La Premier 2026-27 aún no tiene fichero y la ingesta no falla
Equipo sin mapear	Warning y se crea; la ingesta nunca se rompe
Fila de CSV corrupta	Se descarta y se cuenta en <code>skip</code>
Liga sin suficientes partidos	<code>fit_params</code> devuelve <code>None</code> y se registra
Equipo desconocido en predicción	λ de nivel medio de liga, con <code>equipo_local_conocido: false</code> en los parámetros
Partido sin cuotas	Predicción en modo «modelo puro», sin value ni CLV
Fuente sin publicar todavía los resultados	Los partidos siguen en <code>scheduled</code> . <code>GET /api/data-freshness</code> cuenta cuántos partidos ya jugados están pendientes de publicación y hasta qué fecha llega cada liga, y el dashboard lo avisa. No es un fallo del sistema: football-data actualiza cada pocos días
Fichero de temporada aún no publicado (HTTP 300)	La liga aparece sin ningún resultado; se avisa aparte en el dashboard
Marcador demasiado improbable para desglosar	El panel de reparto por tiempos lo dice en vez de mostrar una tabla vacía
Red caída durante la ingesta	3 reintentos con espera creciente; después, warning

Documentación de desarrollo — Quiniela Engine

Cómo trabajar sobre este código: entorno, convenciones, dónde tocar cada cosa y qué hay que verificar antes de dar algo por hecho.

1. Entorno

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe -m pytest          # 165 tests, ~60 s
```

En desarrollo conviene arrancar con recarga en caliente y sin jobs:

```
python run.py --reload --no-scheduler --no-browser
```

`--reload` no abre el navegador solo (uvicorn reinicia el proceso).

Para trabajar con una base de datos aparte sin tocar la buena:

```
set QE_DB_URL=sqlite:///data/pruebas.db
python run.py --setup --no-scheduler
```

2. Mapa del código

Necesito tocar...	Fichero	Cuidado con
Añadir una liga	<code>config.yaml</code> → <code>leagues</code>	El código tiene que existir en football-data. Se siembra en cada arranque
Alias de un equipo	<code>app/ingest/mapper.py</code> → <code>ALIASES</code>	La clave va ya normalizada (minúsculas, sin acentos ni apóstrofes)
Columnas nuevas del CSV	<code>app/ingest/football_data.py</code> → <code>ODDS_BLOCKS</code> / <code>row_hash</code>	Si cambias <code>row_hash</code> , la siguiente ingesta marcará todo como <code>updated</code> (no rompe nada, solo tarda más)
Fórmula del modelo	<code>app/model/dixon_coles.py</code> → <code>_nll_and_grad</code>	Hay gradiente analítico: si cambias la verosimilitud, cambia el gradiente o el ajuste convergerá a cualquier cosa. Ver §4. Ojo con el flag <code>split</code> : hay dos tamaños de vector de parámetros ($3+2n$ y $3+4n$)
Simulación minuto a minuto	<code>app/model/inplay.py</code>	Es la matriz principal por defecto. Si tocas <code>CHASING</code> / <code>LEADING</code> , recuerda que <code>calibrate_state_strength</code> los reescala contra el HT/FT real: cambiar los valores base cambia la escala, no necesariamente el resultado
Rejilla de ξ	<code>config.yaml</code> → <code>tuning.xi_grid</code>	Si el óptimo cae en un extremo de la rejilla, amplíala: es señal de que el óptimo está fuera
Ayudas flotantes (?)	<code>QE.HELP</code> en cada plantilla	El icono se pinta con <code>QE.help('clave')</code> y se activa con <code>QE.initTooltips()</code> después de inyectar el HTML
Nuevos mercados	<code>app/model/markets.py</code> → <code>derive_all</code>	Todo lo que devuelva <code>{p, fair}</code> se pinta solo en la ficha de partido
Qué se guarda en una predicción	<code>app/engine/predictor.py</code> → <code>build_prediction_payload</code>	Es función pura : no toca la base. El backtest la reutiliza
Métricas de evaluación	<code>app/engine/evaluator.py</code>	Añadir columna a <code>Evaluation</code> implica migración (§6)
Endpoints	<code>app/api/routes_data.py</code> / <code>routes_admin.py</code>	Las rutas literales van antes que las paramétricas (<code>/matches/week</code> antes que <code>/matches/{id}</code>)
Páginas	<code>app/web/templates/</code>	Nueva página = ruta en <code>main.py</code> + plantilla que extienda <code>base.html</code>
Estilos	<code>app/web/static/style.css</code>	Variables CSS en <code>:root</code> , sin frameworks

3. Convenciones

- **Castellano** en comentarios, docstrings, nombres de campos de la API y toda la interfaz. El código (nombres de función, variables) en inglés cuando es terminología estándar (`brier` , `logloss` , `score_matrix`).
- **Sin tildes en los ficheros de código** (docstrings y comentarios en ASCII). Los `.md` y las plantillas sí llevan tildes.
- Docstrings que expliquen **por qué**, no qué. Si una línea es rara, el comentario dice el motivo, no la traduce.
- Nada de dependencias nuevas sin una razón que quepa en una frase. La lista actual (`requirements.txt`) es la del SPEC más `requests` , `rapidfuzz` y `httpx` (este último solo para los tests de la API).
- Funciones de `model/` **puras**: entran números, salen números. No reciben `Session` ni leen `config` (salvo defaults). Eso es lo que las hace testeables y reutilizables desde el backtest.

4. Tocar el modelo sin romperlo

Si modificas `_nll_and_grad` , comprueba el gradiente contra diferencias finitas antes de nada:

```
import numpy as np
from app.model.dixon_coales import _nll_and_grad

args = (n_teams, hi, ai, hg, ag, w)
f0, g = _nll_and_grad(x0, *args)
for i in range(len(x0)):
    xp = x0.copy(); xp[i] += 1e-6
    num = (_nll_and_grad(xp, *args)[0] - f0) / 1e-6
    assert abs(num - g[i]) < 1e-3, (i, num, g[i])
```

Después, obligatorio:

```
pytest tests/test_dixon_coales.py
```

Los tests que más protegen son `test_fit_recupera_el_orden_de_fuerza` (datos sintéticos con fuerzas conocidas), `test_fit_respetar_las_restricciones_de_identificabilidad` y `test_dixon_coales_mejora_la_verosimilitud_frente_a_poisson_simple` .

Y una comprobación de negocio que ningún test hace por ti: lanzar un backtest real y mirar el delta de Brier.

```
python -m app.engine.backtest --league SP1 --seasons 4
```

- delta entre -0.005 y $+0.02 \rightarrow$ normal.

- delta muy negativo (el modelo bate al mercado con holgura) → **casi siempre es una fuga de datos**, no una mejora. Búscala antes de celebrar nada.

5. Tests

tests/conftest.py	DB temporal por test + liga sintética (10 equipos, 3 temporadas, goles Poisson con fuerzas conocidas y cuotas coherentes con un margen del 6 %)
tests/test_dixon_coles.py	17 tests: matriz, tau, ajuste, prior, blend, negbin
tests/test_markets.py	18 tests: 1X2, O/U, BTTS, exactos, handicap, tiempos
tests/test_evaluator.py	17 tests: Brier, log-loss, ROI, calibración
tests/test_ingest.py	19 tests: fechas, temporadas, alias, upsert idempotente
tests/test_predictor.py	17 tests: inmutabilidad, payload, value, Kelly
tests/test_backtest_leak.py	7 tests: anti-fuga temporal (los del criterio 6)
tests/test_inplay.py	13 tests: simulación minuto a minuto y xG
tests/test_descomposicion.py	23 tests: reparto 1T/2T, ξ por liga, casa/fuera
tests/test_api.py	34 tests: páginas, endpoints, admin, banner legal, resultados reales y frescura de datos

Los tests **no tocan la red**. La ingesta se prueba con filas de CSV escritas a mano con el formato real.

La `liga_sintetica` es la pieza clave: como los goles se generan con Poisson a partir de fuerzas conocidas, se puede exigir que el ajuste recupere el orden de fuerza y que la calibración sea buena (error máximo < 0.20 por bin), cosa imposible con datos reales.

6. Migraciones de esquema

No hay Alembic (sería desproporcionado). `init_db()` hace `create_all`, que crea tablas nuevas pero **no** añade columnas a tablas existentes.

Para añadir una columna a una tabla que ya existe en producción:

1. Añádela al modelo en `app/models.py`.
2. Añade el `ALTER TABLE` defensivo en `db.py::init_db()`, con comprobación previa sobre `PRAGMA table_info` (o `INFORMATION_SCHEMA` en PostgreSQL) y dentro de un `try`.
3. Si la columna es opcional y la base es pequeña, la alternativa honesta es borrar `data/quiniela.db` y relanzar `run.py --setup` + backfill: son 60 s.

7. Flujo de trabajo típico

Añadir un mercado nuevo (ejemplo: «gana alguna parte»)

1. Función en `markets.py` que reciba las matrices y devuelva `{p, fair}`.

2. Añádela al dict de `derive_all`.
3. Test en `test_markets.py` comprobando que suma 1 y que es coherente con la matriz.
4. La ficha de partido la pinta sola si sigue el formato `{p, fair}`; si quieres sitio propio, añade un `tablaMercado(...)` en `match.html`.
5. Si quieres que se pueda detectar value sobre él, hace falta cuota de mercado: añade la columna en `ODDS_BLOCKS` y el candidato en `value.detect`.

Añadir una liga

1. Añádela a `config.yaml` (o actívala desde `/modelo`).
2. `POST /api/admin/ingest {"league": "I1", "seasons_back": 6}`.
3. Revisa los warnings de `quiniela.mapper` en `logs/quiniela.log`: los equipos sin mapear salen ahí. Añade los alias que hagan falta y vuelve a ingestar.
4. Entrena, predice, y lanza un backtest para ver si el delta de Brier es razonable en esa liga.

8. Logging y diagnóstico

Todo va a `logs/quiniela.log` y a consola.

Logger	Qué cuenta
<code>quiniela.ingest</code>	Filas nuevas / actualizadas / sin cambio por liga y temporada
<code>quiniela.mapper</code>	Equipos sin mapear — lo primero que hay que mirar tras añadir una liga
<code>quiniela.model</code>	Avisos de no convergencia del ajuste
<code>quiniela.predictor</code>	Entrenamientos y predicciones creadas
<code>quiniela.evaluator</code>	Partidos evaluados y errores
<code>quiniela.scheduler</code>	Jobs y su próxima ejecución

Diagnóstico rápido:

```
curl -s localhost:8017/api/health
curl -s localhost:8017/api/stats/summary | python -m json.tool
```

Si un cambio no se refleja: el servidor **no** recarga solo salvo con `--reload`. Reinícialo.

9. Antes de dar algo por terminado

- `[] pytest` en verde (135 tests).

- [] `python run.py --setup` desde cero, sin base de datos previa, funciona.
- [] El dashboard de la semana muestra resultados y resultados posibles.
- [] Un backtest real da un delta de Brier razonable (ni sospechosamente bueno).
- [] Ninguna credencial en el código; todo por `config.yaml` o variables de entorno.
- [] El banner de juego responsable sigue en el pie de todas las páginas (`test_banner_de_juego_responsable_en_todas_las_paginas` lo verifica).
- [] Los DOCs actualizados si has añadido o cambiado funcionalidad.

Importacion de xG (fase 8)

Coloca aqui un fichero por liga con el nombre del codigo de la liga:

```
data/xg/SP1.csv
data/xg/E0.csv
```

Formato (cabecera obligatoria, el orden de columnas da igual):

```
Date,HomeTeam,AwayTeam,xGH,xGA,xGH_HT,xGA_HT
17/08/2026,Barcelona,Valencia,2.31,0.74,1.10,0.35
```

- `Date` en formato dd/mm/aaaa o dd/mm/aa.
- Los nombres de equipo pasan por el mismo normalizador que la ingesta principal, asi que valen las abreviaturas de football-data (`Ath Bilbao` , `Nott'm Forest` ...).
- Las columnas `xGH_HT` y `xGA_HT` (xG de la primera parte) son opcionales.

Despues, en `config.yaml` , activa:

```
xg:
  enabled: true
  weight: 0.35
```

y pulsa "Importar xG" en la pagina Modelo, o `POST /api/admin/xg/import` .

Nota legal: este proyecto NO descarga xG de ningun sitio. El fichero lo pones tu, con datos que tengas derecho a usar. Si tienes un feed propio o de pago, puedes declararlo en `config.yaml` como `xg.sources:` `{SP1: "https://..."}` .

Mejoras y optimizaciones para dar «resultados posibles»

Listado priorizado de lo que haría que la matriz de marcadores y el top-N de resultados posibles fueran **mejores**, no solo más bonitos.

Está ordenado por **relación valor/esfuerzo real sobre este código**, no por lo que suena más sofisticado. Cada punto dice dónde se toca y qué ganancia cabe esperar. Las ganancias están expresadas sobre el estado actual medido:

```
Brier modelo    0.6265 | acierto 1X2      49.1 %
Brier blend     0.6126 | acierto FT top-1 ~11 %
Brier mercado   0.6091 | delta = +0.0174
```

Referencia mental: **bajar 0.005 de Brier ya es una mejora seria**. Cualquier propuesta que prometa 0.05 está mintiendo o tiene fuga de datos.

Estado a 2026-08-23: las mejoras **A3, A4 y A5 ya están implementadas** (marcadas  abajo). Lo que queda pendiente y ordenado por valor es **A1** (lambdas del mercado desde hándicap + total) y **A2** (identidad de equipo entre divisiones).

A. Las cinco que de verdad mueven la aguja

A1. Extraer las lambdas del mercado desde hándicap asiático y línea de goles la mejor

Qué. Ahora el blend usa solo las cuotas 1X2. Pero football-data publica `AHh` (línea de hándicap asiático) y `Avg>2.5 / Avg<2.5`, y **esas dos cosas juntas determinan el par (λ_{local} , $\lambda_{\text{visitante}}$) que está descontando el mercado** con muchísima más precisión que el 1X2:

- la línea de goles fija $\lambda_{\text{local}} + \lambda_{\text{visitante}}$ (total esperado),
- el hándicap fija $\lambda_{\text{local}} - \lambda_{\text{visitante}}$ (diferencia esperada).

Se invierte numéricamente (dos ecuaciones, dos incógnitas) y se obtienen las lambdas implícitas del mercado. Con eso ya no se mezclan solo probabilidades 1X2: se mezclan **lambdas**, y de ahí sale una matriz de marcadores completa calibrada al mercado.

Por qué es la mejor. El 1X2 solo tiene 3 números y pierde toda la información sobre *cómo* se llega a ese resultado. El par hándicap + total es justo lo que hace falta para la matriz. Es la diferencia entre saber «gana el local al 60 %» y saber «se espera un 2.1 – 1.0».

Dónde. `ingest/football_data.py` (guardar `AHh`, `AvgAHH`, `AvgAHA`, `B365AHH/AHA` en `OddsSnapshot` — hoy **se descartan**), módulo nuevo `model/market_lambdas.py`, y `predictor.build_prediction_payload` para mezclar lambdas antes de construir la matriz.

Ganancia esperada. -0.005 a -0.012 de Brier 1X2, y bastante más en marcador exacto y over/under (el mercado de totales es mucho más eficiente que el 1X2). Es la única propuesta de esta lista que puede acercar el sistema al mercado.

Esfuerzo. Medio (1 día). **Riesgo:** bajo, los datos ya se descargan.

A2. Identidad de equipo entre divisiones (ascensos y descensos)

Qué. Ahora `Team` es único **por liga**. El Girona de SP2 y el Girona de SP1 son dos filas distintas sin ninguna relación. Cuando un equipo asciende, el modelo lo trata como recién nacido y le aplica el prior genérico de ascendido, tirando a la basura 38 partidos de información reciente.

La solución no es fusionarlos sin más (una liga es más dura que la otra), sino estimar un **offset de dificultad de división** —a partir de los equipos que han jugado en ambas en los últimos años— y arrastrar el rating del equipo restándole ese offset.

Por qué. En agosto y septiembre, **cada temporada**, hay 3-4 equipos por liga prediciéndose a ciegas. Son exactamente los partidos con más error del año, y son el 10-15 % de los partidos de las primeras jornadas.

Dónde. `models.py` (tabla `club` con relación a `team`, o un campo `global_key` en `Team`), `predictor.fit_params` (arrastre con shrinkage), `dixon-coles.apply_newcomer_prior` (pasa de prior ciego a prior informado).

Ganancia esperada. Poca en el conjunto del año (-0.002 de Brier), pero **grande donde importa**: en las 6 primeras jornadas y en los equipos ascendidos, donde ahora mismo el modelo es casi inútil.

Esfuerzo. Medio-alto (1-2 días, toca esquema). **Riesgo:** medio, hay que migrar la base o recrearla.

A3. HECHO — Simulación minuto a minuto como matriz principal

Qué se hizo. La simulación pasó de bloque informativo a **fuentes de la matriz principal** (`model.primary_matrix: "simulacion"`). Las matrices FT, 1ª y 2ª parte que ve el usuario salen del Monte Carlo minuto a minuto, no de dos Poisson independientes.

Además se resolvió el problema de calibración sin necesitar minutos de gol:

`inplay.calibrate_state_strength` barre una intensidad de dependencia y elige la que reproduce la **distribución HT/FT real de cada liga** (9 celdas, 1 parámetro: sobra información). En LaLiga sale ≈ 0.5 , es decir la dependencia real es **la mitad** de fuerte que los valores de manual: sin calibrar, el modelo la exageraba.

También se añadió `_joint_decomposition`, que guarda el **reparto conjunto** (marcador al descanso | marcador final). Eso es lo que alimenta el desglose 1T/2T al hacer clic en un marcador, y es información que la Poisson independiente sencillamente no puede dar.

Verificación: el backtest puede correr con una u otra matriz (`--no-sim`), y `tests/test_descomposicion.py` comprueba que la dependencia sube las remontadas de forma monótona y que con intensidad 0 se recupera exactamente la independencia.

A4. HECHO — ξ ajustado por liga con validación temporal

Qué. Ahora $\xi = 1.5$ y los límites de p son globales para todas las ligas. No hay ninguna razón para que LaLiga y la Championship tengan la misma memoria: una liga con 24 equipos y 46 jornadas tiene el doble de información por temporada.

Qué se hizo. `app/engine/tuning.py` barre $\xi \in \{0.2 \dots 2.5\}$ por liga con el backtest walk-forward y guarda el ganador en `League.xi`, que `predictor.fit_params` usa a partir de ese momento. Se elige por **log-loss**, no por Brier: el log-loss castiga la sobreconfianza, que es el síntoma de un ξ mal puesto.

Disponible como `POST /api/admin/tune-xi`, como botón en `/modelo` y como `python -m app.engine.tuning --league SP1`.

Resultado real medido. Cada liga quiere una memoria distinta, que era exactamente la hipótesis:

Liga	ξ óptimo	ξ anterior
I1 (Serie A)	0.20	1.5
F1 (Ligue 1)	0.35	1.5
SP1 (LaLiga)	0.50	1.5
E0 (Premier)	1.00	1.5
E1 (Championship)	1.30	1.5

En SP1 la mejora fue de -0.0033 de log-loss y -0.0024 de Brier, en línea con lo previsto. El barrido usa matriz Dixon-Coles a propósito: ξ afecta a las **fuerzas**, no a cómo se convierte λ en matriz, así que tarda segundos en vez de minutos y el orden de los ξ es el mismo.

A5. HECHO — Ataque y defensa separados casa/fuera

Qué. Hoy cada equipo tiene un ataque y una defensa, y la ventaja local y es **la misma para toda la liga**. En la realidad hay equipos que son otra cosa en casa (estadios con ambiente, campos pequeños, altitud) y equipos que se desmoronan fuera.

Modelo extendido: `a_i^casa`, `a_i^fuera`, `d_i^casa`, `d_i^fuera`, con **regularización fuerte** hacia el valor común (`a_i^casa = a_i + \delta_i`, con δ penalizado) para no cuadruplicar la varianza de estimación.

Por qué. Es la extensión clásica del Dixon-Coles y la que más aporta cuando hay suficientes datos. Con 4-6 temporadas por liga los hay.

Qué se hizo. `model.home_away_split: true`. Cada equipo tiene ahora `a ±  $\delta_a$` y `d ±  $\delta_d$` , con penalización L2 de peso 40 sobre los δ . El gradiente analítico se rehízo y **se verificó contra diferencias finitas** (error relativo $2e-5$, ver `docs/DOC_DESARROLLO.md` §4).

La regularización no es decorativa: en el test sintético, con `reg=1` los δ se disparan y con `reg=500` se aplastan a menos de 0.05. El valor 40 detecta un fortín real (δ ataque 0.215 para el equipo que marca un 90 % más en casa) y deja al resto en 0.026 de media.

La media de δ está confundida con γ ; la propia penalización L2 rompe esa degeneración eligiendo la solución de norma mínima. Los equipos con menos de 6 partidos ven su δ anulado proporcionalmente: de un equipo con 3 partidos no se puede decir nada sobre si es mejor en casa.

En la página `/modelo` aparece una columna **fortín** con `$\delta_a + \delta_d$` por equipo.

B. Mejoras de datos (sin datos nuevos no hay milagros)

B1. Alineaciones, bajas y sanciones

Es, con diferencia, **la mayor fuente de error de λ** que queda. Que juegue o no el delantero titular mueve el λ de un equipo más que tres jornadas de forma. football-data no lo publica; requiere una fuente externa (de pago, o una API con licencia). El sistema ya está preparado: sería un ajuste multiplicativo sobre λ antes de construir la matriz, en el mismo punto donde hoy se aplica el xG.

Ganancia potencial: la mayor de toda la lista (-0.01 o más). **Problema:** no hay fuente gratuita legítima. Es una decisión de presupuesto, no de código.

B2. Congestión de calendario y competición europea

Días de descanso desde el partido anterior, y si el equipo jugó competición europea entre semana. **Esto sí es gratis:** se deriva del propio histórico ya ingestado (fechas de partido por equipo) más un calendario europeo.

Efecto conocido: un equipo con 3 días de descanso frente a uno con 7 pierde aproximadamente un 5-8 % de λ ofensivo.

Dónde. Feature nueva en la verosimilitud (un coeficiente global por liga, no por equipo). **Esfuerzo:** medio. **Ganancia:** -0.002 a -0.004 .

B3. xG con una fuente real

El módulo `ingest/xg.py` ya está hecho y probado, pero **está vacío:** no hay fichero en `data/xg/`. El xG es un mejor estimador de la calidad reciente que los goles (menos ruido por partido), sobre todo en ventanas cortas. Conseguir una fuente con licencia y volcarla al CSV es trabajo de gestión, no de programación.

Ganancia esperada: -0.003 a -0.008 , mayor al principio de temporada.

B4. Guardar cuotas en varios momentos, no solo una

Ahora se guarda una foto de cuotas pre-partido y una de cierre. Guardar la evolución (cada ciclo de predicción crea una `OddsSnapshot` nueva con `taken_at`) permitiría medir **el movimiento de línea**, que es información predictiva por sí misma y hace el CLV mucho más honesto.

Esfuerzo: bajo (quitar la restricción única y añadir `taken_at` a la clave). **Ganancia:** no en Brier, sí en calidad del análisis de value.

C. Cómo se miden y se presentan los resultados posibles

C1. Medir con RPS, no solo con Brier y acierto top-1

El **Ranked Probability Score** es la métrica estándar en predicción de fútbol porque penaliza según *lo lejos* que estuvo la predicción, no solo si acertó. Fallar un 1 cuando dabas 40 % al 2 no es lo mismo que fallarlo cuando dabas 5 %.

Y para el marcador exacto, «acertó el top-1» ($\approx 11\%$) es una métrica pobre: mejor **log-score sobre la matriz completa** ($-\log P(\text{marcador real})$), que ya se guarda en `p_assigned_exact_ft` pero no se agrega) y **cobertura del top-N**: qué porcentaje de veces el resultado real está entre los 3 / 5 / 10 más probables. Eso último es lo que el usuario realmente quiere saber al mirar la lista de resultados posibles.

Dónde. `engine/evaluator.py` (nuevas columnas y agregados) + `/stats`. **Esfuerzo:** bajo. **Ganancia:** no mejora el modelo, mejora **saber si el modelo mejora**. Sin esto, las demás propuestas se evalúan a ciegas.

C2. Calibración post-hoc (isotónica) sobre el histórico de evaluaciones

Ya hay 1.800 evaluaciones y una curva de calibración pintada. El siguiente paso es usarla: ajustar una regresión isotónica sobre las probabilidades históricas y aplicarla como corrección final. Si el modelo es sistemáticamente sobreconfiado en favoritos claros (lo habitual), esto lo corrige sin tocar el modelo.

Requisito: entrenar la isotónica **solo con datos anteriores** a cada predicción, o se introduce fuga. El backtest ya tiene la maquinaria.

Esfuerzo: medio. **Ganancia:** -0.002 a -0.005 , casi gratis en cómputo.

C3. Mostrar la incertidumbre de la propia predicción

Ahora se muestra «1-1 al 9.9 %» como si fuera un número exacto. No lo es: sale de unas lambdas estimadas con error. Un bootstrap sobre los parámetros (o el hessiano del ajuste, que L-BFGS-B ya aproxima) daría un intervalo, y permitiría **avisar cuando la predicción es poco fiable** — típicamente equipos recién ascendidos o principios de temporada.

Esfuerzo: medio. **Ganancia:** cero en Brier, mucha en honestidad del producto. Encaja con la filosofía del proyecto.

C4. Agrupar los resultados posibles en escenarios

Un top-10 de marcadores donde el primero tiene 10 % y el décimo 3 % no dice gran cosa. Es más útil presentar **escenarios agrupados**: «victoria local ajustada (1-0, 2-1)», «goleada local (3-0, 3-1, 4-1)», «empate con goles», «empate a cero»... Cada grupo con su probabilidad agregada, que es mucho más alta y más accionable.

Esfuerzo: bajo (es agregación sobre la matriz que ya existe). **Ganancia:** producto, no precisión. Probablemente el cambio de UI con mejor relación esfuerzo/utilidad de la lista.

C5. Ensemble en vez de un solo modelo

Combinar por media geométrica: Dixon-Coles Poisson + Dixon-Coles binomial negativa + simulación minuto a minuto + lambdas del mercado (A1). Los ensembles de modelos correlacionados pero no idénticos casi

siempre calibran mejor que el mejor de sus componentes.

Esfuerzo: bajo, **una vez existan A1 y A3** (las piezas ya estarían). **Ganancia:** -0.002 a -0.004 adicional sobre el mejor componente.

D. Optimizaciones (rendimiento y coste)

D1. Caché de matrices por lambdas redondeadas

`score_matrix` se recalcula para cada partido. Redondeando (λ_{local} , λ_{away} , p) a 3 decimales y cacheando con `functools.lru_cache`, el backtest —que llama miles de veces con lambdas casi repetidas— ahorraría un 30-40 % de tiempo. El error introducido por el redondeo está por debajo de $1e-4$ en probabilidad.

Esfuerzo: 15 minutos. **Ganancia:** backtests más rápidos, más barridos de parámetros posibles.

D2. Vectorizar el backtest por jornada

Ahora se itera partido a partido. Construir las matrices de los 10 partidos de una jornada en un solo `np.einsum` (tensor $10 \times 8 \times 8$) es directo y elimina el overhead de Python. **Ganancia:** $3-5\times$ en el backtest, que es lo que limita cuántas configuraciones se pueden probar (y por tanto A4 y A5).

D3. Adelgazar el JSON de las predicciones

Cada `Prediction` guarda matriz FT (8×8), dos matrices por mitad (6×6), todos los mercados y la simulación: unos 25-30 KB por fila. Con 1.800 filas son ~ 50 MB de SQLite.

Opciones: guardar las matrices como listas de floats de 4 decimales (ya se redondean a 8), o **no guardarlas y recomputarlas** desde λ y p , que es determinista y cuesta microsegundos. Reduciría la base un 80 %.

Esfuerzo: bajo. **Ganancia:** base más pequeña, backups más rápidos, API más ligera.

D4. Simulaciones adaptativas

20.000 simulaciones para todo es exagerado en partidos desequilibrados (donde la distribución converge rápido) y justo en partidos igualados. Criterio de parada por error estándar de la probabilidad de empate: entre 5.000 y 40.000 según haga falta. **Ganancia:** $2-3\times$ en el ciclo de predicción.

D5. Reutilizar el `ModelRun` en el backfill

El backfill crea un `ModelRun` por reajuste (150 filas tras un backfill de 4 ligas). No es un problema de rendimiento, pero ensucia. Marcarlos con un flag `backfill` y poder purgarlos.

E. Lo que NO merece la pena (y por qué)

Idea	Por qué no
Redes neuronales / gradient boosting sobre la matriz	Con ~2.000 partidos por liga y 20-24 equipos no hay datos para que un modelo flexible bata a uno paramétrico bien especificado. Sobreajustará y el backtest lo delatará. Los modelos de goles siguen siendo el estado del arte en este tamaño de muestra
Bivariate Poisson «de verdad» (con componente compartido)	Solo puede modelar correlación positiva entre goles, y la correlación real en fútbol es ligeramente negativa. La corrección τ de Dixon-Coles ya captura mejor el fenómeno real, que es un exceso de marcadores bajos
Meteorología, árbitro, distancia de viaje	Efectos por debajo del ruido. Se ha medido muchas veces: no sobreviven a una validación temporal honesta
Más ligas para «tener más datos»	El modelo se ajusta por liga . Añadir la Superliga griega no mejora ni un poco la predicción de LaLiga. Solo tiene sentido para transferencia entre divisiones del mismo país (A2)
Subir el tamaño de la matriz de 8×8 a 12×12	La masa por encima de 7 goles es < 0.1 %. Se redistribuye proporcionalmente y no afecta a ningún mercado real
Kelly completo en vez de fraccional	Con probabilidades imperfectas (y lo son: el delta de Brier lo demuestra) Kelly completo produce rachas de pérdida brutales. Ya está topado a 1/4 y al 2 % del banco a propósito

Plan sugerido, en orden

Fase	Qué	Estado
~~2~~	A4 (ajustar ξ por liga con el backtest)	✓ hecho — cada liga con su ξ
~~4~~	A3 (simulación minuto a minuto como matriz principal)	✓ hecho — y calibrada contra HT/FT real
~~8~~	A5 (casa/fuera regularizado)	✓ hecho — gradiente verificado
1	C1 (medir bien: RPS, log-score de marcador, cobertura top-N)	pendiente, 3 h. Parcialmente cubierto: el contraste por partido ya muestra el puesto del resultado real
3	A1 (lambdas del mercado desde hándicap + total)	pendiente — la mejor que queda , 1 día, -0.012
5	C2 (calibración isotónica) + C4 (escenarios en la UI)	pendiente, 1 día
6	D1 + D2 (caché y vectorización)	pendiente, 4 h
7	A2 (identidad entre divisiones)	pendiente, 2 días

Con A1 y C2 encima de lo ya hecho, el delta contra el mercado debería quedar del orden de **+0.005**, es decir a la altura del mercado.

Nunca por debajo del mercado de forma sostenida. Si un backtest dice que sí, lo primero que hay que buscar es la fuga de datos, no la botella de champán. El propio backtest ya avisa cuando eso pasa.